

Claude Code 입문 · 하네스부터 forge까지

AI 코딩, 한 단계 위로

프롬프트를 치는 사람에서, 루프를 설계하는 사람으로

왜 지금 이것을 알아야 하나

AI에게 더 잘 “부탁”하는 법이 아니라, AI를 “굴리는 시스템”을 짜는 법.

프롬프트 한 줄을 다듬는 것만으로 생산성이 갈리는 시대는 지나가고 있습니다.

오늘의 여정

기초 → 사고방식 → 메커니즘 → 종합

1

기초

Claude Code 기본 기능 — 우리가 다룰 도구

2

사고방식 (왜)

하네스 · 루프 · 컴파운드 엔지니어링

3

메커니즘 (어떻게)

그릴링 · 에이전트 · Dynamic Workflow

4

종합

forge — 위 전부를 묶은 한국어 워크플로우

01

Claude Code 기본 기능

터미널에 상주하는 AI 코딩 에이전트



Claude Code란?

터미널에서 `claude` 명령으로 실행하는 대화형 AI 코딩 에이전트.
코드베이스 전체에 접근해 파일을 읽고·고치고·명령을 실행한다.

- 내가 타이핑하는 게 아니라 — 위임하고 검토한다
- CLI · 데스크톱(Mac/Win) · 웹 · IDE 확장(VS Code·JetBrains) 동일 엔진

비유 IDE 속 자동완성이 아니라, “터미널에 상주하는 시니어 동료.” 파일을 직접 열어보고, 명령을 돌려 결과를 확인하며 일한다.

핵심 기능 지도 — “사무실” 비유

각 기능을 회사 조직에 빗대면 한눈에 들어온다



Skills

업무 매뉴얼 — 반복 절차를 SKILL.md로 패키징(필요할 때만 로드)



CLAUDE.md

사내 규정집 — 팀 표준·개인 선호를 세션마다 기억



Hooks

자동 결재 규칙 — 이벤트마다 셸 명령 실행(포맷·린트·알림)



Permissions

출입 권한 — 도구 사용을 Allow/Ask/Deny로 통제



MCP

외부 거래처 직통 회선 — GitHub·Jira·DB 표준 연결



Subagents

전담 인턴 — 무거운 탐색·검증을 격리해 위임

대화 · 명령 · 계획

매일 쓰는 세 가지 조작.

- 대화형 CLI — `claude`로 코드베이스 맥락 위에서 자연어로 작업
- Slash commands — `/init` `/model` `/context` `/agents` `/mcp`로 세션 제어
- Plan mode — 읽기만 하고 계획만 세우는 모드(소스 미수정), 큰 변경 전 안전장치

팁 Plan mode는 “먼저 보고 받고, 승인 후 실행” — 위험한 변경을 시작 전에 차단한다.

기억하는 방법 — Skills & CLAUDE.md

사람은 잊지만, 잘 쓰면 시스템은 기억한다.

- Skills — 반복 작업을 절차로 패키징. 필요할 때만 lazy-load돼 컨텍스트 절약
- CLAUDE.md(Memory) — 계층형 영구 지시(managed→user→project→local). 팀 표준·개인 선호 보존

왜 중요한가 이 “기억”이 뒤에 나올 ‘컴파운드 엔지니어링’(학습을 복리로 쌓기)의 토대가 된다.

자동화와 통제 — Hooks & Permissions

에이전트를 믿되, 검증 가능하게.

- Hooks — 생명주기 이벤트(PreToolUse·PostToolUse·SessionStart 등)에 셸 명령 실행
- 결정론적 자동화 — 자동 포맷·린트·권한 검증·알림을 ‘사람이 아니라 시스템’이 보장
- Permissions — Allow/Ask/Deny + 권한 모드(plan·acceptEdits·bypass)로 도구 사용 통제

왜 중요한가 ‘무인 자동화는 무인 실수’ — Hooks·Permissions가 그 위험을 시스템 차원에서 줄인다(Part 2 하네스의 검증·가드레일과 직결).

위임과 연결 — Subagents & MCP

혼자 다 하지 않는다. 나눠 맡기고, 밖과 잇는다.

- Subagents — 독립 컨텍스트를 가진 전담 작업자. 큰 탐색·검증을 격리해 메인을 보호
- MCP(Model Context Protocol) — 외부 도구(GitHub·Slack·DB)를 표준으로 직접 연결

예고 Subagents는 Part 3에서 '에이전트'로, MCP는 외부 연동의 표준으로 다시 등장한다.

“CLAUDE.md에 다 적으면 똑똑해진다” — 틀렸습니다.

- CLAUDE.md는 매 세션 컨텍스트에 상주 — 길수록 토큰을 먹고 집중도가 떨어진다
- 공식 권장: 200줄 이하 + 구체적 지시
- 반복 절차는 lazy-load되는 Skills로 빼는 게 정석

02

사고방식 왜 이렇게 일하는가

하네스 · 루프 · 컴파운드 엔지니어링



하네스 엔지니어링

LLM을 '일하는 에이전트'로 만드는 — 둘러싼 실행 계층 전체.

- 모델을 반복 호출하고, 출력을 파싱해 도구를 실행하고, 결과를 다시 넣는 '루프'
- 컨텍스트·메모리·권한·상태를 관리하고, 언제 멈출지 결정하는 '스캐폴딩'

핵심 테제 (MongoDB Eng.) “LLM은 당신의 에이전트 시스템에서 가장 작은 부품이다.” — 성능의 대부분은 모델이 아니라 하네스가 좌우한다.



비유 — 드라이버와 레이싱카

“

아무리 뛰어난 F1 드라이버(LLM) 여도,
차·타이어·텔레메트리·피트크루(하네스)가 형편없으면 못 이
긴다.

우승은 차고(harness) 엔지니어링에서 갈린다.

하네스의 구성요소

이 여섯 가지를 설계하는 것이 '하네스 엔지니어링'



실행 루프

호출 → 파싱 → 도구 실행 → 결과 주입 →
멈춤 판단



도구 디스패치

어떤 도구를 언제 부르고 실패를 어떻게
회복할지



컨텍스트 관리

윈도우 압축(compaction) — 무엇을 남기고
버릴지



상태·메모리

checkpoints·중간 결과를 디스크에 영속화



가드레일

권한·안전 불변식 — 위험한 동작을 막는다



검증·관측

결과가 맞는지 확인하고 추적(tracing)

“더 좋은 모델로 바꾸면 해결된다” — 대개 틀렸습니다.

- 실무에서 에이전트가 헤매는 원인은 대부분 컨텍스트·도구·검증 같은 하네스 결함
- 모델 IQ가 아니라 ‘둘러싼 시스템’을 고쳐야 풀린다
- 레버리지: 프롬프트 튜닝보다 하네스를 설계하는 쪽이 훨씬 크다



루프 엔지니어링 (하네스의 대표 실천)

“

“루프 엔지니어링이란, 에이전트에게 프롬프트를 치는 사람
자리에서 스스로 물러나
대신 그 일을 하는 시스템을 당신이 직접 설계하는 것이다.”

— Addy Osmani, “Loop Engineering” (2026-06-07)

루프 엔지니어링의 프리미티브

Osmani 원문: 5개 core + State/Memory — (정확 표기)



Automations

스케줄로 돌며 스스로 발굴·트리아지



Worktrees

병렬 에이전트가 서로 충돌하지 않게 격리



Skills

에이전트가 추측할 프로젝트 지식을 적어 둔다



Plugins/Connectors

이미 쓰는 도구에 에이전트를 연결



Sub-agents

하나는 안을 내고, 다른 하나가 검증한다



State/Memory

대화 밖에 사는 파일/보드 — 무엇이 끝났고 다음은 무엇인지

루프의 그림자 — Osmani의 경고

루프는 검증 책임을 없애지 않는다. 오히려 증폭시킨다.

- 토큰 비용 — 사용 패턴에 따라 비용이 크게 출렁인다
- 이해 부채 — 내가 안 쓴 코드가 빨리 쌓일수록 ‘이해의 빗’이 커진다
- 인지적 항복 — “그냥 주는 대로 받자”는 유혹
- 오케스트레이션 세금 — 병렬 수를 정하는 건 도구가 아니라 ‘내 리뷰 대역폭’

결론 “당신의 일은, 작동을 확인한 코드를 출하하는 것이다.” — 그냥 시작 버튼을 누르는 사람이 아니라 끝까지 엔지니어로 남아라.



컴파운드 엔지니어링

“

“각 엔지니어링 작업 단위는 다음 단위를
더 어렵게가 아니라 — 더 쉽게 만들어야 한다.”

— Kieran Klaassen / Every · 학습을 복리로 쌓는 자기개선형 개발

복리는 ‘마지막 한 단계’에서 돈다

학습을 명시적으로 문서·룰로 환원할 때만



메모리를 가진 시스템

PR이 가르치고, 버그가 영구 교훈이 되고, 리뷰가 기본값을 갱신



실패 → 규칙

버그 → 테스트 + CLAUDE.md 규칙 + eval 추가로 같은 부류 차단



계획 우선

“계획이 새로운 코드다” — 구현 전 상세 계획, 병렬 전문 리뷰



50/50 규칙

절반은 기능, 절반은 시스템(룰·문서·자동화) 개선

흔한 오해 “AI에게 많이 시키면 저절로 복리” — 아니다. ‘학습 환원’ 단계를 빼면 그냥 빠른 단리.

세 사고방식, 한 줄로

하네스

모델이 아니라 '둘러싼 시스템'이 성능을 만든다

루프

프롬프트에서 물러나 '루프 설계자'가 된다 (단, 검증 책임은 남는다)

컴파운드

매 작업이 다음을 더 쉽게 — 학습을 복리로 쌓는다

→ 그럼 이 사고방식들을 '실제로' 굴리는 메커니즘은? → Part 3

03

메커니즘 — 어떻게 굴리는 가

그릴링 · 에이전트 · Dynamic Workflow





그릴링 (Grilling)

“

“계획이나 설계를 날카롭게 버리는 집요한 인터뷰 —
그 과정에서 문서(ADR과 용어집)까지 함께 만들어낸다.”

— Matt Pocock, grill-with-docs

그릴링은 왜 효과적인가

가장 비싼 버그는 '서로 다른 걸 같은 단어로 부른' 데서 나온다.

- 용어집 대조 — 기존 정의와 충돌하면 즉시 지적
- 모호어 정밀화 — 'account'가 Customer인지 User인지 못박는다
- 구체 시나리오 — 엣지케이스를 발명해 경계를 강제로 정밀화
- 코드와 대조 — 말과 코드가 어긋나면 표면화
- 결정을 ADR·용어집으로 — 휘발되지 않고 다음 작업의 연료가 된다



그릴링 비유 — 반대신문

“

변호사의 반대신문처럼 — “대충 그렇다”를 용납하지 않고
한 질문씩 파고들어 모순을 드러낸다.

흔한 오해: “트집·시간낭비”? 아니다 — 썩 단계에서 비싼 모호함을 죽이는 것. (사
소한 일엔 과하니 가볍게)

에이전트 (Subagents)

혼자 다 보지 않는다 — 독립된 작업자에게 위임하고 ‘요약’만 받는다.

- 독립 컨텍스트 — 메인 대화를 로드하지 않아, 큰 탐색 로그가 메인을 오염시키지 않음
- 역할 카드 = `.claude/agents/<name>.md` — description의 ‘when to use’로 위임 판단
- 도구 제한·모델 오버라이드·병렬 실행 — 탐색/리뷰를 동시에 fan-out

오해 주의 “서브에이전트는 메인 대화를 다 안다” — 정반대. 독립 컨텍스트라 ‘명시적으로 넘긴 것’만 안다.



maker / checker 분리 — “자기 채점 금지”

“

작가와 편집자가 같은 사람이면 안 된다.
자기 글의 오타는 안 보인다 — 다른 눈이 봐야 잡힌다.

독립 checker가 ‘반증(refute)’ 자세로 보면, maker가 놓친 실패모드를 잡는다.



Dynamic Workflow

“

“Dynamic Workflow는 서버에이전트를 대규모로 오케스트레이션하는 JavaScript 스크립트다. Claude가 스크립트를 작성하고, 런타임이 세션이 계속 응답하는 동안 백그라운드에서 실행한다.”

Claude Code 공식 기능 · 2026-05-28 출시(v2.1.154+)

대화형 에이전트 vs 워크플로우 — “누가 계획을 주는가”

	대화형 (Subagents/Teams)	Dynamic Workflow
다음에 뭘 할지 결정	Claude가 매 턴 판단	스크립트가 (코드로)
중간 결과 위치	Claude 컨텍스트에 쌓임	스크립트 변수에
규모	턴당 소수	런당 수십~수백
재현성	매번 즉흥	같은 공정 그대로 재실행

워크플로우가 주는 것

단지 '많이 돌리기'가 아니라, '신뢰성 패탄'을 코드로 박는 것.

- fan-out / pipeline — 수십~수백 병렬 + 단계별 검증 게이트
- 어드버서리얼 교차검증 — 독립 에이전트가 서로의 발견을 반증
- 동시 16개 · 런당 1,000개 상한 — 폭주 방지
- 재개 가능(resumable) — 완료분은 캐시, 나머지만 재실행

실제로 이게 되나? — Bun 포팅

99.8%

기존 테스트 통과

~75만 줄

생성된 Rust 코드

11일

첫 커밋 → 머지

Jarred Sumner — Dynamic Workflow로 Bun을 Zig → Rust 포팅

흔한 오해 “많이 돌리니 무조건 좋다” — 토큰 비용이 크다. 큰 작업 전 작은 슬라이스로 비용을 가늠하라.

메커니즘 3종 → forge로

그릴링

→ fg-ask (계획을 도메인에 대고 심문)

에이전트

→ fg-agents (역할 카드) + fg-run/fg-adversarial-review (maker/checker)

Dynamic Workflow

→ fg-run (계획을 워크플로우로 실행)

→ 이 부품들을 한 루프로 엮으면? → forge

04

forge — 전부를 한 루프로

오늘 배운 사고방식과 메커니즘의 한국어 구현체



forge란?

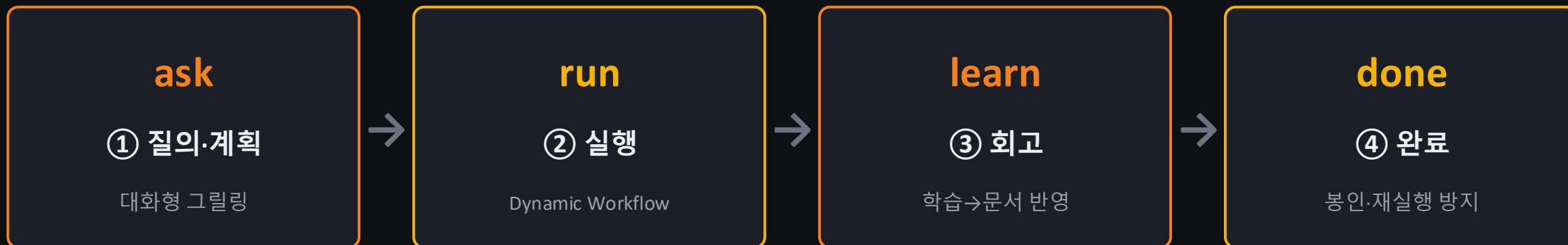
작업 하나를 '질의·계획 → 실행 → 회고 → 완료'의 한 바퀴로 돌리는 개발 루프.

- 18개 fg- 프리픽스 Claude Code 스킬 — 루프 4단계 + 루프 밖 유틸리티 14개
- 계획은 대화형 그릴링, 실행은 Dynamic Workflow, 회고는 학습을 문서로 환원
- 완료 단계가 작업을 '봉인' — 같은 작업이 두 번 실행되지 않게

사실 3개만 /fg-ask → /fg-run → /fg-next — 평소엔 이 셋으로 굴린다.

forge 루프 — 4단계

한 스킬이 끝나면 다음으로 가는 길을 ‘알리고 멈춘다’



↳ done 이 봉인하면 — 새 작업으로서만 ask 에서 다시 시작

두 기둥 그릴링·회고는 워크플로우 ‘밖’ 대화·문서는 산출물이 아니라 루프의 연료

forge의 두 기둥

왜 이렇게 설계했는가



그릴링은 워크플로우 ‘밖’ 대화로

Dynamic Workflow는 실행 중 사용자 입력을 못 받는다 — 그래서 한 질문씩 그릴링은 루프 밖 대화로 둔다



문서는 산출물이 아니라 ‘연료’

계획에서 다듬은 용어가 실행의 기준이 되고, 회고의 학습이 다음 계획의 출발점이 된다

forge README “문서는 루프의 연료다 — 부산물이 아니라.”

오늘 배운 전부 → forge

하네스 엔지니어링



.forge/ 상태계약 + 4단계 루프 + fg-doctor

루프 엔지니어링



fg-loop · 신뢰 사다리 L1→L2→L3(forge 프레이밍)

컴파운드 엔지니어링



fg-learn (회고→문서 승급), '문서는 연료'

그릴링



fg-ask

에이전트 (maker/checker)



fg-agents + fg-run / fg-adversarial-review

Dynamic Workflow



fg-run · fg-adversarial-review

forge의 강점 — 경고를 ‘게이트’로

Osmani가 권고한 경고들을, forge는 권고가 아니라 강제 게이트로 만든다.

- 검증 없이는 봉인 불가 — 검증 결정이 기록 안 되면 done 불가
- 회고가 기본값 — 건너뛰기는 계획과 실제 결과의 차이가 작을 때만, 감사 가능하게
- 활성 슬롯 항상 1개 — 루프 폭을 사람의 리뷰 대역폭에 맞춘다
- eco 모드 — 비용 출렁임에 대비해 서브에이전트 모델 복잡도 절감

실전 — 사실 셋만 쓰면 된다



신뢰 사다리 L1 관찰(fg-status) · L2 보조(fg-next) · L3 무인(fg-next all / fg-loop) — (forge 프레이밍)

언제 무엇을 쓰나

처음 셋업

fg-map → fg-agents → fg-ask → 루프

일상 작업

fg-ask → fg-run → fg-next

사소한 1회 변경

fg-quick (가볍게 그릴링, 바로 실행)

무인 주행

fg-ask × N → fg-next all · 목표수렴: fg-loop

재진입 / 점검

fg-status (어디까지) · fg-doctor (상태 건강?)

한 문장으로

forge는 오늘 배운 '전부'를
하나의 반복 가능한 루프로 묶은 것.

하네스 + 루프 + 컴파운드 × 그릴링 + 에이전트 + 워크플로우

기억할 세 문장

“

에이전트는 잊어도, 저장소는
기억한다.

— Addy Osmani

“

계획이 새로운 코드다.

— Kieran Klaassen

“

문서는 루프의 연료다 — 부산
물이 아니라.

— forge

루프를 만들어라 — 그러나 엔지니어로 남아라.

그냥 ‘시작 버튼’을 누르는 사람이 아니라, 끝까지 검증하는 엔지니어로 남아라.

\$ 시작: `/plugin marketplace add gyuha/forg` → `/plugin install forge@forge`

\$ 그다음: `/fg-ask` 한 뒤 `/fg-next` (또는 `/fg-next all`)을 계속

감사합니다 · Q&A